

1 Two Sum — LeetCode Problem #1

Problem Statement:

Given an array of integers `nums` and an integer `target`, return indices of the two numbers such that they add up to `target`.

- Assume that each input has exactly **one solution**.
 - **Do not reuse** the same element twice.
 - You can return the answer in any order.
-

📥 Examples

Example 1:

Input: `nums = [2,7,11,15]`, `target = 9`
Output: `[0,1]`
Explanation: `nums[0] + nums[1] == 9`

Example 2:

Input: `nums = [3,2,4]`, `target = 6`
Output: `[1,2]`

Example 3:

Input: `nums = [3,3]`, `target = 6`
Output: `[0,1]`

✓ Constraints

- $2 \leq \text{nums.length} \leq 10^4$
 - $-10^9 \leq \text{nums}[i] \leq 10^9$
 - $-10^9 \leq \text{target} \leq 10^9$
 - Only one valid answer exists.
-

💡 Follow-up:

Can you solve it in **less than $O(n^2)$** time complexity?

🚀 Optimal Solution: HashMap — $O(n)$

```

from typing import List

class Solution:
    def twoSum(self, nums: List[int], target: int) -> List[int]:
        num_map = {} # number -> index

        for i, num in enumerate(nums):
            complement = target - num
            if complement in num_map:
                return [num_map[complement], i]
            num_map[num] = i

# Example
sol = Solution()
result = sol.twoSum([2, 7, 11, 15], 9)
print(result) # Output: [0, 1]

```

🔗 Step-by-Step Execution:

```

nums = [2, 7, 11, 15], target = 9
num_map = {}

```

```

Iteration 1: i = 0, num = 2
→ complement = 9 - 2 = 7
→ 7 not in map → add 2:0 → {2: 0}

```

```

Iteration 2: i = 1, num = 7
→ complement = 9 - 7 = 2
→ 2 in map → return [0, 1]

```

✓ Final Output: [0, 1]

⬅️ END Ends early when pair is found.

❑ Way 1: Brute Force — $O(n^2)$

```

class Solution1:
    def twoSum(self, nums: List[int], target: int) -> List[int]:
        for i in range(len(nums)):
            for j in range(i + 1, len(nums)):
                if nums[i] + nums[j] == target:
                    return [i, j]

```

❑ Dry Run:

```

nums = [2, 7, 11, 15], target = 9

```

```

i = 0, j = 1
2 + 7 = 9 ✓ → return [0, 1]

```

✓ Pros:

- Simple, easy to understand
- ✗ Cons:
- Very slow for large inputs

□ Way 2: Two Pointer Approach — $O(n \log n)$

```
class Solution:
    def twoSum(self, nums: List[int], target: int) -> List[int]:
        nums_with_index = [(num, i) for i, num in enumerate(nums)]
        nums_with_index.sort()

        left, right = 0, len(nums) - 1
        while left < right:
            sum_val = nums_with_index[left][0] + nums_with_index[right][0]
            if sum_val == target:
                return [nums_with_index[left][1],
                        nums_with_index[right][1]]
            elif sum_val < target:
                left += 1
            else:
                right -= 1
```

🔍 Dry Run

nums = [2, 7, 11, 15], target = 9
 With index → [(2, 0), (7, 1), (11, 2), (15, 3)]

Start:
 left = 0 → 2
 right = 3 → 15
 sum = 17 ✗ → right = 2
 sum = 13 ✗ → right = 1
 sum = 9 ✓ → return [0, 1]

✓ Pros:

- Faster than brute force
- Good when returning values or working with sorted arrays
- ✗ Cons:
- Requires sorting and index tracking

🏁 Summary of All Approaches

Approach	Time Complexity	Space Complexity	Notes
Brute Force	$O(n^2)$	$O(1)$	Simple but slow
Two Pointers	$O(n \log n)$	$O(n)$	Needs sorting, track original indices

Approach	Time Complexity	Space Complexity	Notes
HashMap ✓	$O(n)$	$O(n)$	Best and most optimal solution

Best Time to Buy and Sell Stock

Problem Statement:

You are given an array `prices` where `prices[i]` is the price of a given stock on the i -th day.

Your goal is to **maximize your profit** by choosing a **single day to buy one stock** and a **different day in the future to sell that stock**.

Return the **maximum profit** you can achieve from this transaction.
If no profit is possible, return **0**.

Examples

Example 1:

Input: `prices = [7,1,5,3,6,4]`

Output: 5

Explanation: Buy on day 1 (price = 1), sell on day 4 (price = 6), profit = $6 - 1 = 5$.

Example 2:

Input: `prices = [7,6,4,3,1]`

Output: 0

Explanation: No profitable transaction is possible.

Constraints

- $1 \leq \text{prices.length} \leq 10^5$
 - $0 \leq \text{prices}[i] \leq 10^4$
-

Python Code — Optimal Solution (One Pass)

```
from typing import List
```

```
class Solution:
```

```
    def maxProfit(self, prices: List[int]) -> int:
```

```
        min_price = float('inf') # Start with a very large number
```

```
        max_profit = 0
```

```
        for price in prices:
```

```
            if price < min_price:
```

```
                min_price = price # Found a lower price to buy
```

```
            elif price - min_price > max_profit:
```

```
                max_profit = price - min_price # Better profit found
```

```
return max_profit
```

✔ Time and Space Complexity

Metric Complexity

Time $O(n)$

Space $O(1)$

□ Problem (In Simple Words)

You are given a list of stock prices for n days:

```
prices = [7, 1, 5, 3, 6, 4]
```

Each value represents the stock price on that day:

Day	Price
0	₹7
1	₹1 ✔ (best day to buy)
2	₹5
3	₹3
4	₹6 ✔ (best day to sell)
5	₹4

Best transaction: Buy on day 1 at ₹1 and sell on day 4 at ₹6 → **Profit = ₹5**

🔄 Dry Run: Execution Table

Day	Price	min_price	price - min_price	max_profit	Explanation
0	7	7	-	0	Initialize min_price
1	1 ✔	1 ✔	-	0	New lower price
2	5	1	4 ✔	4	Profit = 5 - 1
3	3	1	2	4	Less than max_profit
4	6 ✔	1	5 ✔	5 ✔	Best profit
5	4	1	3	5	Less than max

✔ Line-by-Line Explanation

```
min_price = float('inf') # Start with no minimum
```

```
max_profit = 0                # No profit yet

for price in prices:          # Go through each day's price
    if price < min_price:
        min_price = price      # Better day to buy
    elif price - min_price > max_profit:
        max_profit = price - min_price  # Better day to sell

return max_profit
```

🎯 Key Idea

Always track the **lowest price so far** to determine when it's the best day to buy. Then compare it with future prices to find the **maximum profit** you can get.

You only loop **once** → $O(n)$ time and $O(1)$ space — ✔ Very efficient.

Best Time to Buy and Sell Stock II

Problem Statement

You are given an integer array `prices` where `prices[i]` is the price of a given stock on the i -th day.

On each day, you **may choose to buy and/or sell** the stock.

You can **only hold at most one share** at any time.

However, you can buy it and immediately sell it **on the same day**.

Return the maximum profit you can achieve.

Examples

Example 1:

Input: `prices = [7,1,5,3,6,4]`

Output: 7

Explanation:

Buy on day 1 (price = 1), sell on day 2 (price = 5) $\rightarrow$ profit = 4

Buy on day 3 (price = 3), sell on day 4 (price = 6) $\rightarrow$ profit = 3

Total Profit = 4 + 3 = 7

Example 2:

Input: `prices = [1,2,3,4,5]`

Output: 4

Explanation: Buy on day 0, sell on day 4 $\rightarrow$ profit = 5 - 1 = 4

Example 3:

Input: `prices = [7,6,4,3,1]`

Output: 0

Explanation: No profitable transactions possible.

Constraints

- $1 \leq \text{prices.length} \leq 3 \cdot 10^4$
 - $0 \leq \text{prices}[i] \leq 10^4$
-

Python Code (Greedy Approach)

```
from typing import List

class Solution:
    def maxProfit(self, prices: List[int]) -> int:
        profit = 0
        for i in range(1, len(prices)):
            if prices[i] > prices[i - 1]:
                profit += prices[i] - prices[i - 1]
        return profit
```

Time and Space Complexity

Metric	Complexity
Time Complexity	$O(n)$
Space Complexity	$O(1)$

Intuition and Step-by-Step Breakdown

◆ Line 1:

```
for i in range(1, len(prices)):
```

- Start from index 1 because we are comparing today's price with **yesterday's**.
 - We skip the first day since there is no previous day to compare.
-

◆ Line 2:

```
if prices[i] > prices[i - 1]:
```

- If today's price is **greater than yesterday's**, we buy yesterday and sell today to make a profit.
-

◆ Line 3:

```
profit += prices[i] - prices[i - 1]
```

- We **calculate profit** and add it to the running total.
 - This simulates **multiple small transactions** rather than one big one.
-

↩ Final Line:

```
return profit
```

- Return the **total accumulated profit** after checking all days.
-

★ Example Walkthrough

Input:

```
prices = [1, 5, 3, 6]
```

Day (i)	prices[i-1]	prices[i]	Is i > i-1?	Profit Added	Total Profit
1	1	5	✓ Yes	5 - 1 = 4	4
2	5	3	✗ No	0	4
3	3	6	✓ Yes	6 - 3 = 3	7

✓ **Final Output:** 7

□ Key Insight

This is a **greedy approach** —

we grab **all the small profitable opportunities** to maximize total return.

🔄 217. Contains Duplicate

■ Problem Statement

Given an integer array `nums`, return `true` if **any value appears at least twice**, and `false` if **every element is distinct**.

⬇ Examples

Example 1:

Input: `nums = [1,2,3,1]`

Output: `true`

Explanation: The element 1 appears at indices 0 and 3.

Example 2:

Input: `nums = [1,2,3,4]`

Output: `false`

Explanation: All elements are unique.

Example 3:

Input: `nums = [1,1,1,3,3,4,3,2,4,2]`

Output: `true`

✓ Constraints

- $1 \leq \text{nums.length} \leq 10^5$
 - $-10^9 \leq \text{nums}[i] \leq 10^9$
-

📄 Python Code

```
from typing import List
```

```
class Solution:
```

```
    def containsDuplicate(self, nums: List[int]) -> bool:
```

```
        seen = set()
```

```
        for num in nums:
```

```
            if num in seen:
```

```
                return True
```

```
            seen.add(num)
```

```
        return False
```

🔄 219. Contains Duplicate II

■ Problem Statement

Given an integer array `nums` and an integer `k`, return `true` if there are **two distinct indices `i` and `j`** such that:

- `nums[i] == nums[j]`
 - `abs(i - j) <= k`
-

⬇ Examples

Example 1:

Input: `nums = [1,2,3,1]`, `k = 3`
Output: `true`

Example 2:

Input: `nums = [1,0,1,1]`, `k = 1`
Output: `true`

Example 3:

Input: `nums = [1,2,3,1,2,3]`, `k = 2`
Output: `false`

✓ Constraints

- `1 <= nums.length <= 105`
 - `-109 <= nums[i] <= 109`
 - `0 <= k <= 105`
-

💻 Python Code

```
from typing import List

class Solution:
    def containsNearbyDuplicate(self, nums: List[int], k: int) -> bool:
        index_map = {} # Stores the last seen index of each number

        for i, num in enumerate(nums):
            if num in index_map:
                if i - index_map[num] <= k:
                    return True
            index_map[num] = i
```

```
return False
```

🔍 How It Works

We use a dictionary `index_map` to store the **last seen index** of each number.

For each number:

- If it was seen before:
 - Check the distance between current index and previous one.
 - If $\text{distance} \leq k$, return `True`
 - Otherwise, update its last seen index.
-

📖 Example 1:

```
nums = [1, 2, 3, 1], k = 3
```

Index	Value	Seen Before	Last Index	Distance	Action
0	1	✗ No	-	-	Save 1 → map[1]=0
1	2	✗ No	-	-	Save 2 → map[2]=1
2	3	✗ No	-	-	Save 3 → map[3]=2
3	1	✓ Yes	0	3	✓ return True

📖 Example 2:

```
nums = [1, 0, 1, 1], k = 1
```

Index	Value	Seen Before	Last Index	Distance	Action
0	1	✗ No	-	-	Save 1 → map[1]=0
1	0	✗ No	-	-	Save 0 → map[0]=1
2	1	✓ Yes	0	2	✗ > k → update 1 →
2					
3	1	✓ Yes	2	1	✓ return True

📖 Example 3:

```
nums = [1, 2, 3, 1, 2, 3], k = 2
```

Index	Value	Seen Before	Last Index	Distance	Action
0	1	✗ No	-	-	map[1] = 0
1	2	✗ No	-	-	map[2] = 1
2	3	✗ No	-	-	map[3] = 2

3	1	✓ Yes	0	3	✗ > k → update 1 →
3					
4	2	✓ Yes	1	3	✗ > k → update 2 →
4					
5	3	✓ Yes	2	3	✗ > k → update 3 →
5					

✗ No duplicates within distance $\leq k \rightarrow$ **return False**
